

CS.105 Programming Foundations Practical 3

The aim of this practical is to introduce you to Java source code and class definitions.

Please ask a demonstrator in the drop in lab if you need some help with Practical 3.

Both parts A and B are set to be undertaken during your week 3 lab time. Part A should be undertaken before part B.

Part A

Fun with Ticket Machines

First download and unzip projects from CS105 MyPlace corresponding to those in chapter 2 of the textbook.

In BlueJ open the **naive-ticket-machine** project (from the above projects). Create a couple of ticket machine objects and try to obtain a good understanding of the ticket machines' behaviour by interacting with them on the object bench.

Examining a class definition

Take a look at the source code for the ticket machine class by double clicking its icon in the class diagram. This will display the Java source code in a separate window.

Class Header and Fields

1. Does it matter whether we write `public class TicketMachine` or `class public TicketMachine` in the outer wrapper of a class? Edit the source of the `TicketMachine` class to make the change and then close the editor window. Do you notice a change in the class diagram? What error message do you get when you now ask to compile? Do you think this message clearly explains what is wrong? (Reminder: to compile, right click on class icon and choose compile, or pick Compile from left of screen or select from Tools menu.)

Change the class back to how it was, making sure this clears the error when you compile it.

2. In the following field declaration from the `TicketMachine` class, `private int price;` does it matter which order the three words appear in? Edit the `TicketMachine` class to try different orderings, asking to compile to see if there is an error message.

Make sure that you reinstate the original version after your experiments. (You can always get the original projects' zip file back from MyPlace).

3. Is it always necessary to have a semicolon at the end of a field declaration? Once again, experiment via the editor and then asking to compile. The rule you will learn here is an important one, so be sure to remember it. (Note: you can also ask to compile within the Editor window.)

Accessor methods

4. If the name of `getBalance` is changed to `getAmount`, does the return statement in the body of the method need to be changed too? Try it out within BlueJ.
5. Add in an accessor method, `getTotal`, that returns the value of the total field.
6. Try removing the `return` statement from the body of `getPrice`. What error message do you see now when you try compiling the class? Put the return statement back. If you have forgotten, it was `return price;`

Mutator methods

7. Complete the body of the `setPrice` method below so that it assigns the value of its parameter to the price field. Add the method in to your class.

```
public void setPrice(int cost)
{
    // Complete the body of the setPrice method here
}
}
```

- Complete and add in the following method, whose purpose is to subtract the value of its parameter from the field named price.

```
public void discount(int amount)
{

}

}
```

Printing from Methods

- Add a method called **prompt** to the **TicketMachine** class. This should have a **void** return type and take no parameters. The body of the method should print something like:
Please insert the correct amount of money.
- Add a **showPrice** method to the **TicketMachine** class. This should have a **void** return type and take no parameters. The body of the method should print something like:
The price of a ticket is xyz cents.
where xyz should be replaced by the value held in the price field when the method is called.
- Create two ticket machine objects with differently priced tickets. Do calls to their **showPrice** methods show the same output, or different? How do you explain this effect?

More on the naive ticket machine

Before attempting these exercises, be sure that you have a good understanding of how the ticket machines behave, and how that behaviour is implemented through the fields, constructor and methods of the class.

- Give the class an additional constructor, so that one should take a single parameter that specifies the price, and the other should take no parameter and set the price to be a default value of your choosing. Test your implementation by creating machines via the two different constructors.
- Implement a method, **empty**, that simulates the effect of removing all money from the machine. This method should have a **void** return type, and its body should simply set the **total** field to zero. Does this method need to take any parameters? Test your method by creating a machine, inserting some money, printing some tickets, checking the total, emptying the machine and then checking the total. Is the empty method a mutator or an accessor?

Part B

- Look at the menus available in BlueJ and create a new project called Air-Conditioner. (When prompted with a folder you should add **\ Air-Conditioner** to the end of the path name where you keep your CS105 projects.)
- Look at the menus and create a new class called AirCon within your Air-Conditioner project.

Open the editor on the AirCon class. The class header will have been produced automatically for you within the code, along with some other template code which you will need to edit, as described below.

Comments in the source code are ignored by the compiler. Comment lines are enclosed in **/**** (or **/***) and ***/**. Text to the right of **//** is also a comment.

- Replace the field declaration with the declaration of a field which will hold the desired temperature, giving it an appropriate name and Java type. It should be able to hold an integer. (Note: your code will not compile at this point as you will also need to remove further reference to variable x in the code which follows, as described below at steps 4 and 5)
- Update the code so that a newly created AirCon object would have its desired temperature set to 20. (Remove mention of variable x.)
- Replace the sample method in the code with one which returns the value of the desired temperature. (Think carefully about appropriate parameters, return type and return value.)
- Add a further method which causes the desired temperature to be increased from its current value by the fixed amount 2.
- Add a further method which causes the desired temperature to be decreased from its current value by the fixed amount 2.
- Add a further method which causes the desired temperature to be increased by the amount passed as parameter.

9. Include a method which will print the desired temperature along with some appropriate accompanying text to the terminal window.
10. Add in another field which will store whether or not the air conditioning unit is switched on or off. The type should be boolean and you should think carefully of a meaningful name for the field.
11. Update your constructor so that it also initialises the air conditioning unit to be switched on.
12. and 13. Add two further methods: one to switch the air conditioning unit on and the other to switch it off.
14. Update your method which prints to the terminal window, so that it also reports on whether the unit is on or off.
(Note: you have not yet been taught about 'if' statements early in week 3 and are not expected to use an 'if' statement here, although you may if you wish). The output line in this task could look as follows in the terminal window, depending on whether the unit is on or off:
"It is true that the air conditioning unit is switched on"
or
"It is false that the air conditioning unit is switched on"
Hint: make use of +

Reminder: help with practical 3 is available during the optional drop in lab and on mattermost.